I've renamed the `build` task to `build:js` to make it parallel with `build:css`, but the big change is the new `dev` command that calls tsc-watch.

I'm calling tsc-watch with four arguments:

- `noClear`, which prevents tsc-watch from clearing the console window. (I'd like to do that myself, thank you very much.)
- `-p tsconfig.json`, which points to the TypeScript configuration file that governs the compilation I want to do
- `--onSuccess \"yarn build:js\"`, which controls what you want to have happen if the TypeScript compilation succeeds. In our case, we want the regular esbuild `build:js` to happen, since we now know the code is type-safe.
- `--onFailure \"yarn failure:js\"`, which controls what you want to have happen if the TypeScript compilation fails. I guess I had options here, but what I chose to do is the `failure.js` script, namely `rm ./app/assets/builds/application.js && rm ./app/assets/builds/application.js.map`, meaning removing the existing esbuild files from the build directory so that the development browser page will error rather than return the most recent successful compilations. I thought that allowing the most recent success to stick around would be confusing.

To get this to work, we need the `dev` command to replace `build.js` in the `Procfile`:

chapter_01/03/Procfile.dev

```
web: bin/rails server -p 3000
js: yarn dev
css: yarn build:css --watch
```

And that works. `yarn dev` calls tsc-watch, which sets itself up as a watcher automatically, so we don't need to do that again here.

When a relevant file changes, tsc-watch is triggered and runs the TypeScript compiler. If the compile is successful, and only if it is successful, esbuild is called upon to bundle the code into a browser-friendly form. If the compile fails,